

系统开发工具基础 · 实验报告（第 2 周）

姓名：甘如嫣 学号：25090032014

2026 年 9 月 8 日

授课内容： 命令行环境；开发环境与工具；Debugging and Profiling。

1 GitHub 仓库与提交记录

本次课程的**报告、练习、代码**均已上传至个人 GitHub 仓库，采用**增量提交**（分多次 commit，禁止单次全量提交）：

<https://github.com/ganruyan/system-basics-tools>

本周（第 2 周）的提交按内容拆分为 8 次独立 commit，例如：

```
1 bc6a7a8 chore: 忽略 IDE 和 Python 缓存目录
2 a34cda7 week2: 第2周实验报告 (LaTeX 源码 + PDF)
3 83bb696 week2: q05 信号与任务控制 (worker.sh)
4 dcf669a week2: q06 语义重构
5 984ab11 week2: q07 归并排序调试修复
6 c7ce5ed week2: q08 词频程序优化
7 99f403f week2: 各题完整指令与课后练习 11 实例
8 cc06b2f fix(week2): 修正 q07 测试期望值笔误 (多一个 4 致 1 failed)
```

每次提交只针对一个逻辑单元（一个题、一份报告或一项配置），保留了清晰的提交历史，便于追踪与回滚。GitHub 上的 commit 记录截图如下：

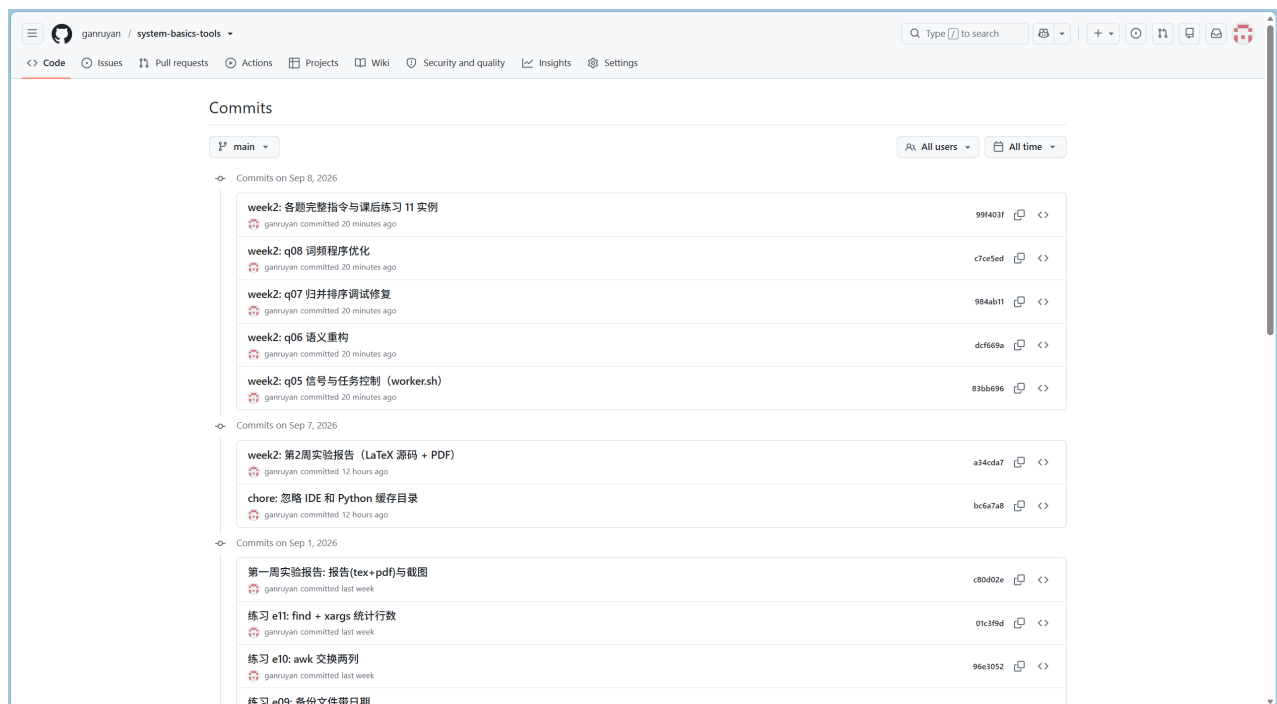


图 1: GitHub 仓库 commit 记录 (ganruyan/system-basics-tools, main 分支)

2 第 5 题 控制一个可清理的后台任务

主题：命令行环境：进程、信号与任务控制 **建议用时：** 12–15 分钟

实验目的：掌握前台/后台任务切换、挂起 (Ctrl-Z)、jobs/bg 任务控制，以及用信号优雅终止可清理的任务。

实验步骤：将给定脚本保存为 q05/worker.sh 后执行：

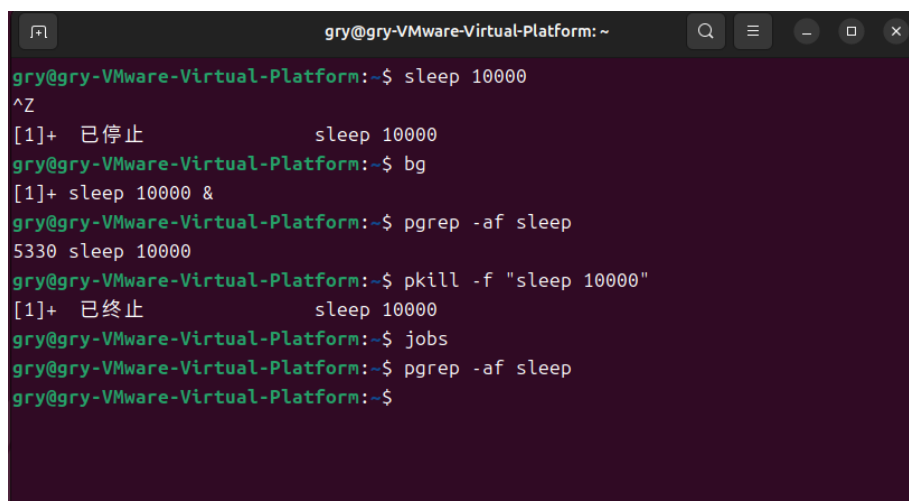
```
1 cd q05
2 chmod +x worker.sh
3 ./worker.sh > stdout.log 2> stderr.log      # 前台启动，stdout/stderr 分开重定向
4 # 按 Ctrl-Z 挂起前台任务
5 bg                                           # 放入后台继续运行
6 jobs                                         # 确认状态：[1]+ Running ...
7 PID=$(jobs -p)                              # 取得 PID (不手抄)
8 kill -TERM "$PID"                           # 发送 SIGTERM
9 wait "$PID" 2>/dev/null                     # 等待任务结束
10 cat cleanup.log                             # 应出现 CLEAN_EXIT
```

实验结果： jobs 显示任务处于 Running 状态；发送 SIGTERM 后任务退出，cleanup.log 中出现 CLEAN_EXIT；全程未使用 kill -9。

```
gry@gry-VMware-Virtual-Platform:~/globtest$ mkdir -p ~/q05 && cd ~/q05
gry@gry-VMware-Virtual-Platform:~/q05$ cat > worker.sh << 'EOF'
> #!/usr/bin/env bash
> trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
> n=0
> while true; do
> echo "$n"
> n=$((n+1))
> sleep 1
> done
> EOF
gry@gry-VMware-Virtual-Platform:~/q05$ chmod +x worker.sh
gry@gry-VMware-Virtual-Platform:~/q05$ ls -l worker.sh
-rwxrwxr-x 1 gry gry 128  9月  7 16:16 worker.sh
gry@gry-VMware-Virtual-Platform:~/q05$ ./worker.sh 1>stdout.log 2>stderr.log
^Z
[1]+  已停止                  ./worker.sh > stdout.log 2> stderr.log
gry@gry-VMware-Virtual-Platform:~/q05$ bg
[1]+  ./worker.sh > stdout.log 2> stderr.log &
gry@gry-VMware-Virtual-Platform:~/q05$ jobs
[1]+  运行中                  ./worker.sh > stdout.log 2> stderr.log &
gry@gry-VMware-Virtual-Platform:~/q05$ kill -TERM $(jobs -p)
gry@gry-VMware-Virtual-Platform:~/q05$ wait %1
[1]+  已完成                  ./worker.sh > stdout.log 2> stderr.log
gry@gry-VMware-Virtual-Platform:~/q05$ cat cleanup.log
CLEAN_EXIT
gry@gry-VMware-Virtual-Platform:~/q05$ jobs
gry@gry-VMware-Virtual-Platform:~/q05$ cat stdout.log
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
```

图 2: 第 5 题: worker.sh 启动、bg/jobs 任务控制与 CLEAN_EXIT 输出

说明: 脚本顶部的 `trap '...' TERM INT` 捕获终止信号, 先写入 `cleanup.log` 再 `exit 0`, 实现优雅退出。`jobs -p` 直接返回任务 PID, 避免手工抄写带来的误差。补充练习中还通过 `sleep 10000` 配合 `pgrep/pkill` 验证了按名称查找并终止后台任务:



```
gry@gry-VMware-Virtual-Platform: ~  
gry@gry-VMware-Virtual-Platform:~$ sleep 10000  
^Z  
[1]+ 已停止                  sleep 10000  
gry@gry-VMware-Virtual-Platform:~$ bg  
[1]+ sleep 10000 &  
gry@gry-VMware-Virtual-Platform:~$ pgrep -af sleep  
5330 sleep 10000  
gry@gry-VMware-Virtual-Platform:~$ pkill -f "sleep 10000"  
[1]+ 已终止                  sleep 10000  
gry@gry-VMware-Virtual-Platform:~$ jobs  
gry@gry-VMware-Virtual-Platform:~$ pgrep -af sleep  
gry@gry-VMware-Virtual-Platform:~$
```

图 3: 补充: sleep 10000 后台任务 + bg/jobs/pgrep/pkill 控制

3 第 6 题 语义重构与本地开发反馈

主题: 开发环境与工具 **建议用时:** 12-15 分钟

实验目的: 使用编辑器的 Python 语言服务器完成跳转、查找引用与符号重命名,并用 ruff 与 pytest 获得本地反馈。

实验步骤:

1. 在 q06 中按题目创建 math_utils.py、app.py、test_math_utils.py 三个文件。
2. 在 PyCharm 中打开 q06, 启用 Python 语言服务器。
3. 对 total_price 分别执行**跳转到定义** (Ctrl+B) 与**查找引用** (Alt+F7)。
4. 使用**重命名符号** (Shift+F6) 把 total_price 改为 calculate_total, 定义与两处引用同步更新。
5. 在 app.py 中临时加入未使用的 import os, 用 ruff 检测并删除该行。
6. 运行 ruff check 与 pytest, 保证全部通过。

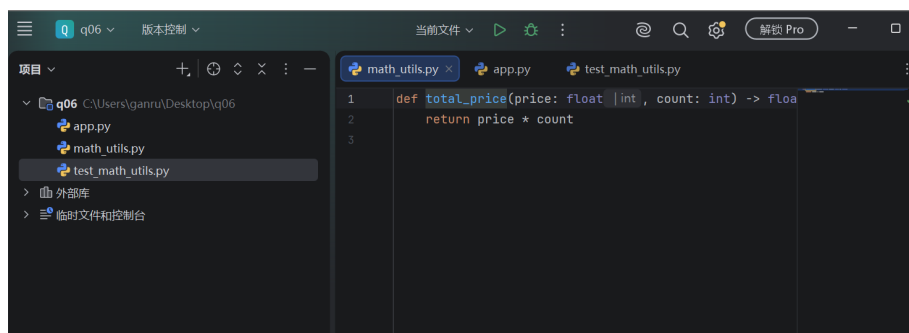


图 4: 第 6 题: q06 中三个文件 math_utils.py/app.py/test_math_utils.py

跳转到定义与查找引用：光标置于 `total_price` 上，`Ctrl+B` 跳到 `math_utils.py` 中的定义，`Alt+F7` 列出全部引用位置。

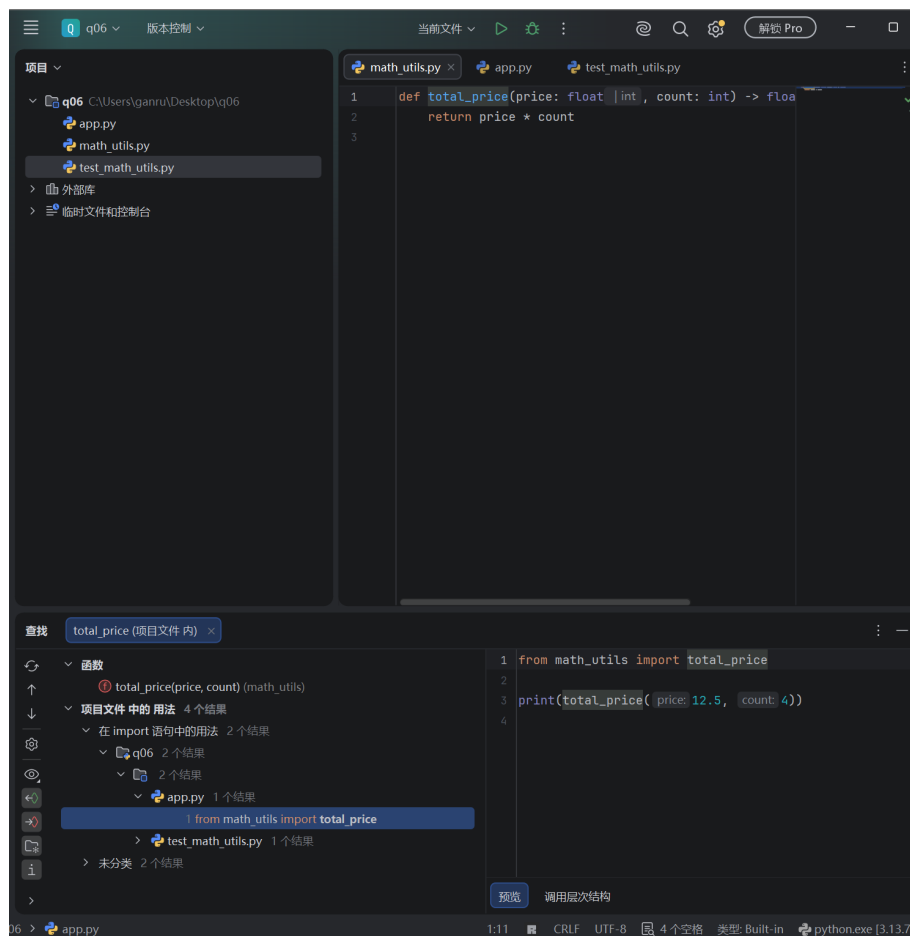


图 5: `total_price` 跳转到定义 / 查找引用（语言服务器）

重命名符号：`Shift+F6` 把 `total_price` 重命名为 `calculate_total`，三处（定义 + 两处引用）同步更新：

```
1 # math_utils.py
2 def calculate_total(price: float, count: int) -> float:
3     return price * count
4
5 # app.py
6 from math_utils import calculate_total
7 print(calculate_total(12.5, 4))
8
9 # test_math_utils.py
10 from math_utils import calculate_total
11 def test_total_price():
12     assert calculate_total(12.5, 4) == 50.0
```

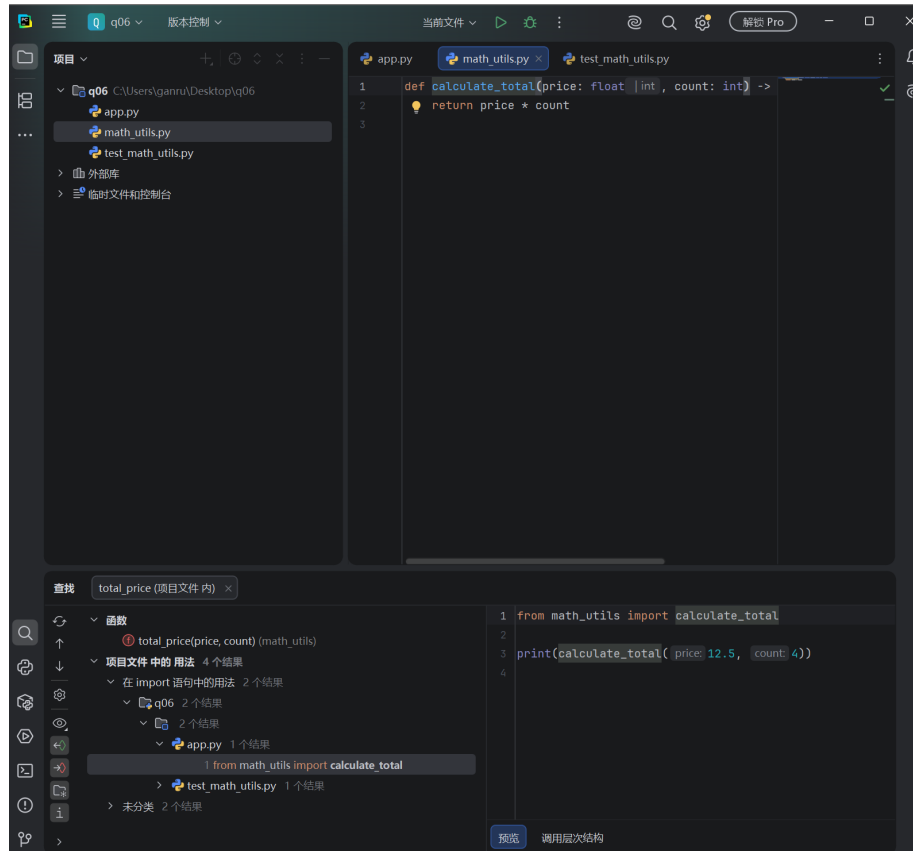


图 6: 重命名符号后 `total_price` → `calculate_total` 同步生效

实验结果: 加入未使用的 `import os` 后, `ruff check` 报错 (`F401 os imported but unused`); 删除该行后 `ruff check` 无告警; `pytest` 输出 `1 passed`。

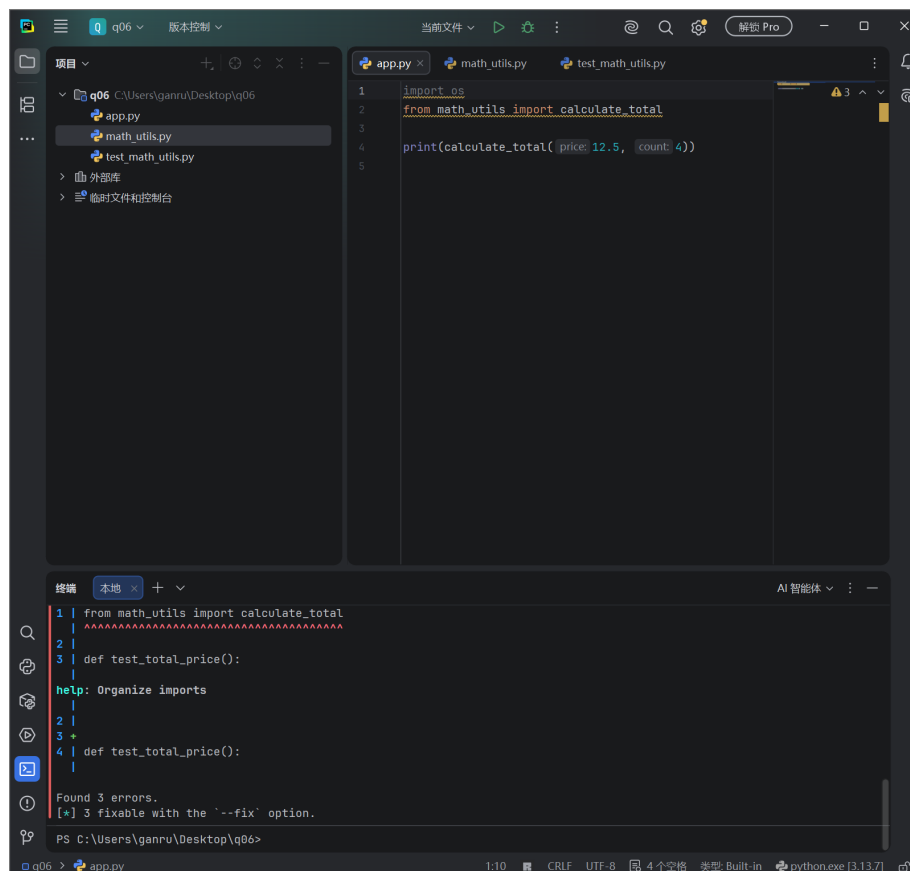


图 7: ruff check 报出未使用的 import os

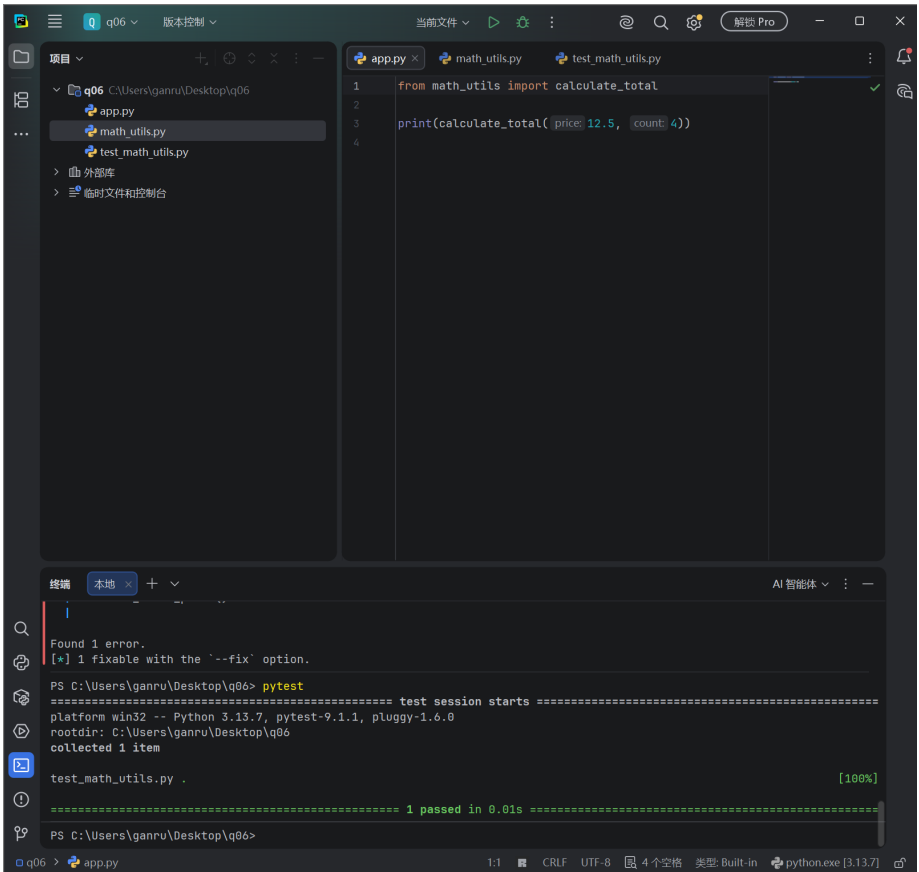


图 8: pytest 输出 1 passed in 0.01s

说明：符号重命名是语义级重构，由语言服务器统一改写定义与所有引用，避免手工遗漏。未使用的导入由 ruff（基于 AST 的 linter）发现并删除，保持代码整洁。

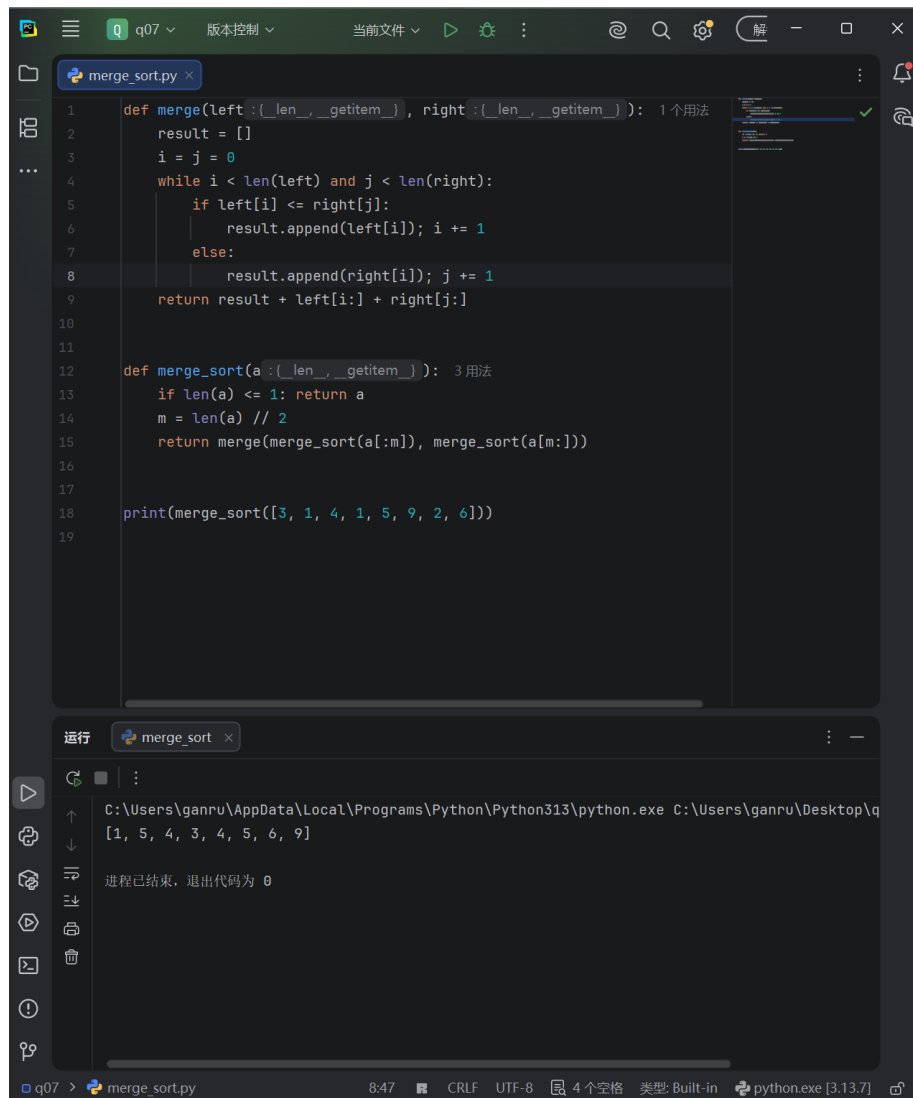
4 第 7 题 用调试器定位归并排序缺陷

主题： Debugging **建议用时：** 12–15 分钟

实验目的： 使用调试器定位并修复归并排序中的下标缺陷，并补充测试保证正确性。

实验步骤：

1. 将缺陷代码保存为 q07/merge_sort.py 并运行，输出 [1, 5, 4, 3, 4, 5, 6, 9]，与正确的 [1, 1, 2, 3, 4, 5, 6, 9] 不符。
2. 在 merge 函数设断点，单步观察 i、j、left、right 的取值变化。
3. 定位缺陷：else 分支误用 right[i]，应为 right[j]。
4. 完成最小修复（不替换为 sorted）。

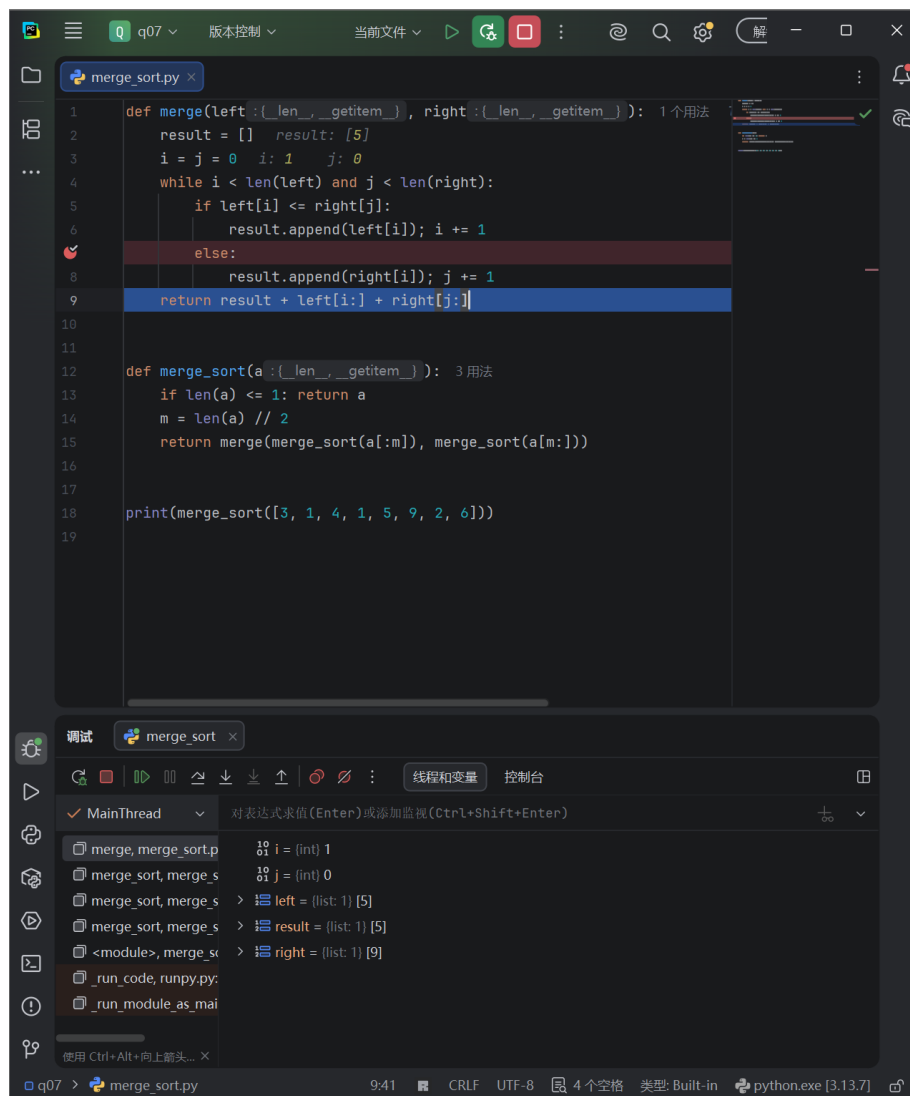


The image shows a Python IDE window with a file named `merge_sort.py`. The code implements a merge sort algorithm. The `merge` function takes two lists, `left` and `right`, and merges them into a new list `result`. The `merge_sort` function recursively splits the input list `a` into halves until it reaches a base case of length 1, then merges the sorted halves back together. The main execution line is `print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))`.

```
1 def merge(left : (_len_, __getitem__), right : (_len_, __getitem__)): 1 个用法
2     result = []
3     i = j = 0
4     while i < len(left) and j < len(right):
5         if left[i] <= right[j]:
6             result.append(left[i]); i += 1
7         else:
8             result.append(right[j]); j += 1
9     return result + left[i:] + right[j:]
10
11
12 def merge_sort(a : (_len_, __getitem__)): 3 用法
13     if len(a) <= 1: return a
14     m = len(a) // 2
15     return merge(merge_sort(a[:m]), merge_sort(a[m:]))
16
17
18 print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
19
```

The bottom panel shows the execution output in the "运行" (Run) window. The command executed is `C:\Users\ganru\AppData\Local\Programs\Python\Python313\python.exe C:\Users\ganru\Desktop\q07.py`. The output is `[1, 5, 4, 3, 4, 5, 6, 9]`, which is the result of the merge sort algorithm applied to the input list `[3, 1, 4, 1, 5, 9, 2, 6]`. The status bar at the bottom indicates the file is `merge_sort.py`, the editor is at line 8, column 47, and the Python interpreter is `python.exe [3.13.7]`.

图 9: 第 7 题: 缺陷代码输出乱序结果 [1, 5, 4, 3, 4, 5, 6, 9]

图 10: 调试器断点观察变量: `else` 分支应取右侧指针 `j` 对应的元素

缺陷位置与修复:

```

1 while i < len(left) and j < len(right):
2     if left[i] <= right[j]:
3         result.append(left[i]); i += 1
4     else:
5         result.append(right[j]); j += 1 # 原为 right[i], 已修复

```

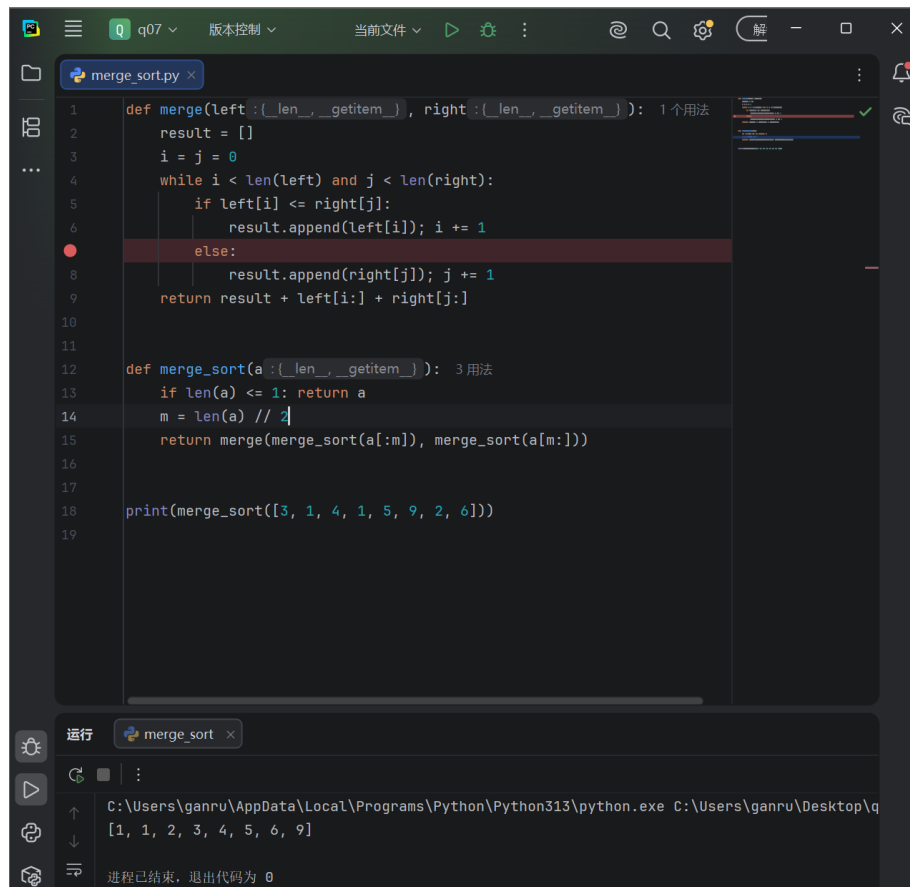


图 11: 修复后输出正确结果 [1, 1, 2, 3, 4, 5, 6, 9]

补充测试:

```
1 # test_merge_sort.py
2 from merge_sort import merge_sort
3
4 def test_given_input():
5     assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1, 1, 2, 3, 4, 5, 6, 9]
6
7 def test_duplicates():
8     assert merge_sort([5, 3, 5, 1, 3, 1]) == [1, 1, 3, 3, 5, 5]
```

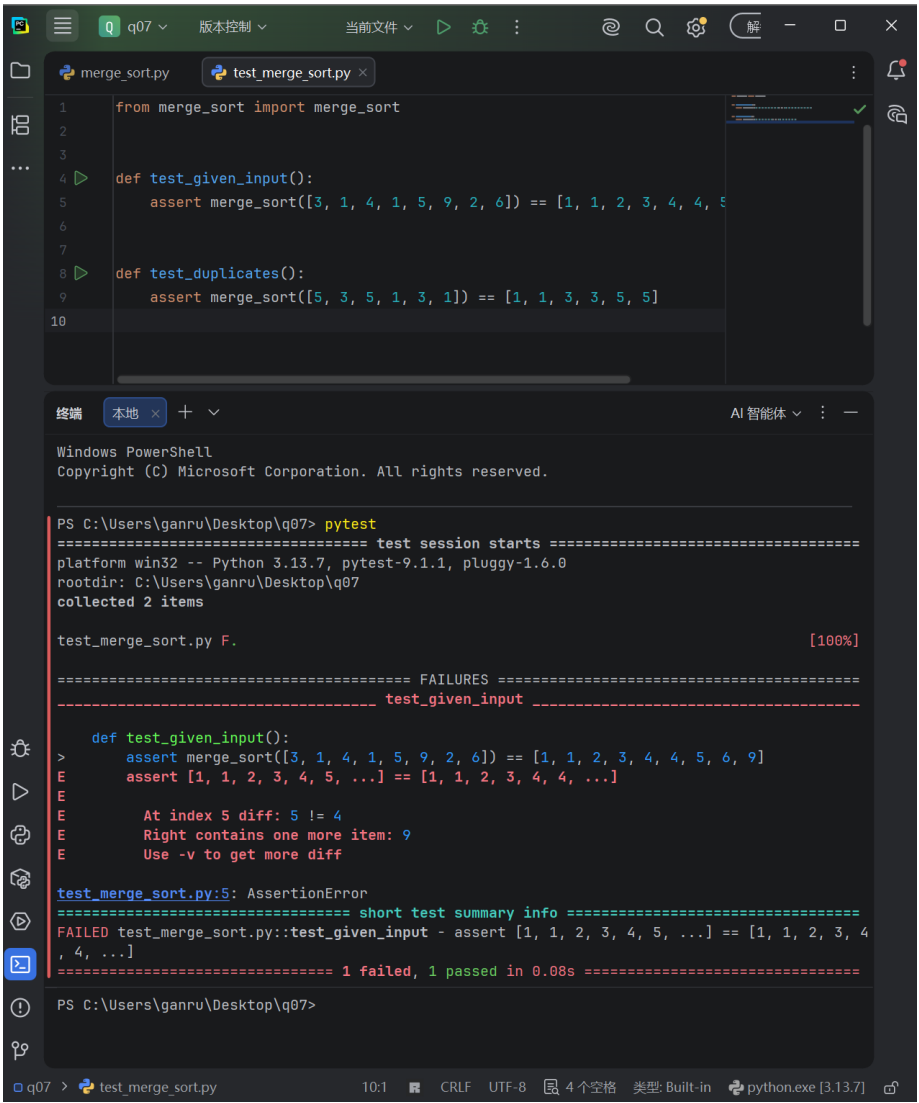


图 12: pytest 运行两个测试（图为首轮：test_given_input 因期望值笔误失败，修正后为 2 passed）

实验结果：修复后运行输出正确；pytest 输出 2 passed；未使用 sorted 替换归并排序。

说明：else 分支对应“取右侧较小元素”，其下标应为右侧指针 j；误用 i 既可能越界，也会产生错误结果。调试时借助断点与变量面板，定位到错误发生的确切一行，比反复 print 更高效。

5 第 8 题 先测量，再优化慢速词频程序

主题：Profiling **建议用时：**12–15 分钟

实验目的：用计时与 cProfile 先测量、定位瓶颈，再对故意低效的去重程序做优化，并量化加速效果。

实验步骤：

```
1 cd q08
2 python generate_words.py # 生成 words.txt（可重复）
```

```
3 Measure-Command { python wordfreq.py }           # Windows 下用 Measure-Command 计时 (等价 time)
4 Measure-Command { python wordfreq.py }           # 运行 2 次取中位数
5 python -m cProfile -s cumulative wordfreq.py      # 定位耗时函数/行
```

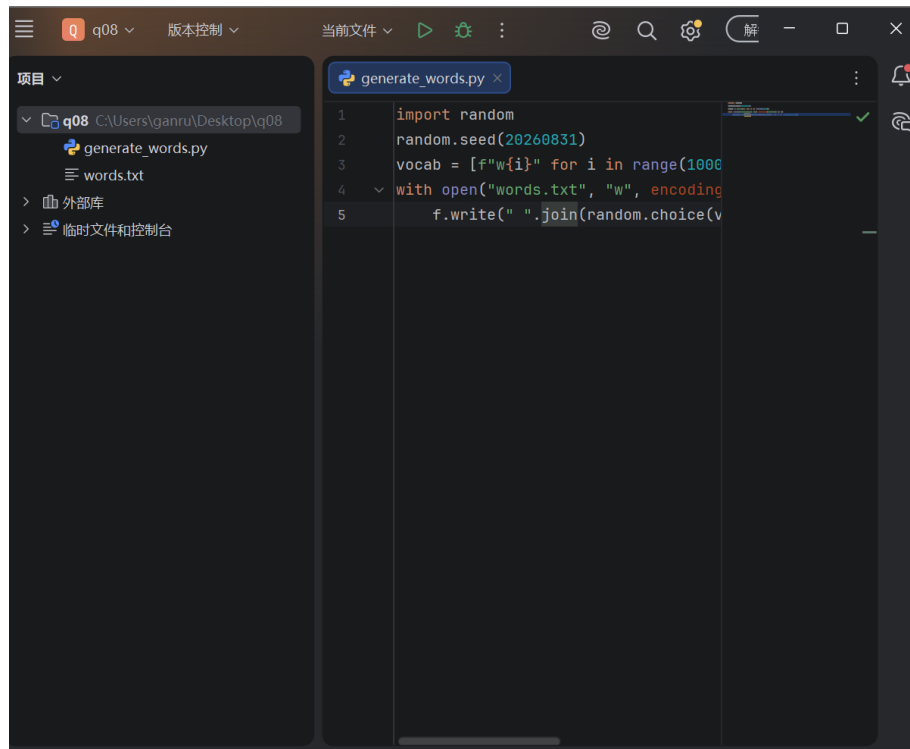


图 13: 第 8 题: generate_words.py 生成 words.txt

原始 (低效) 程序用列表判重, count 输出为 1000:

```
1 words = open("words.txt", encoding="utf-8").read().split()
2 unique = []
3 for word in words:
4     if word not in unique:      # 列表线性查找, O(n)
5         unique.append(word)
6 print("count=", len(unique))
```

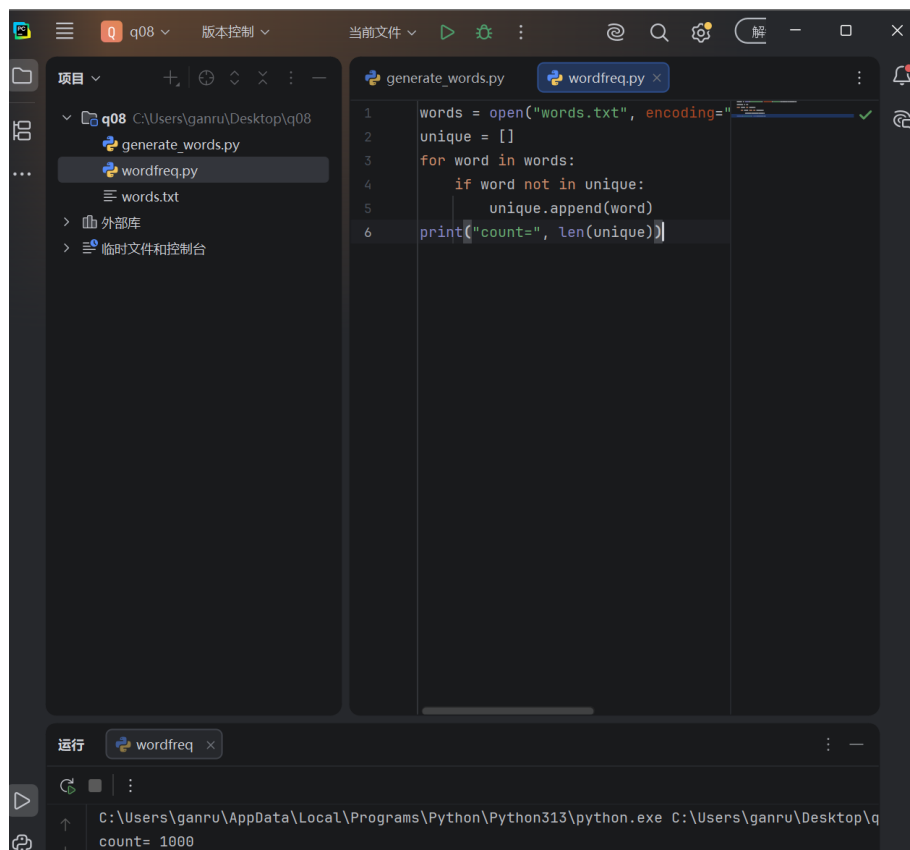


图 14: 原始程序输出 count= 1000

定位结果：cProfile -s cumulative 输出显示，主要耗时集中在 if word not in unique——在列表 unique 上做线性查找（list.append 与成员判断），整体复杂度为 $O(n^2)$ 。

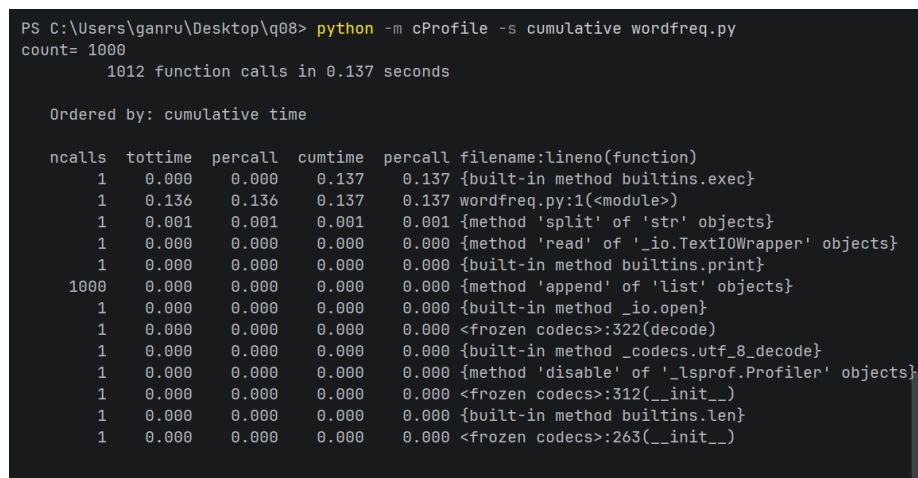
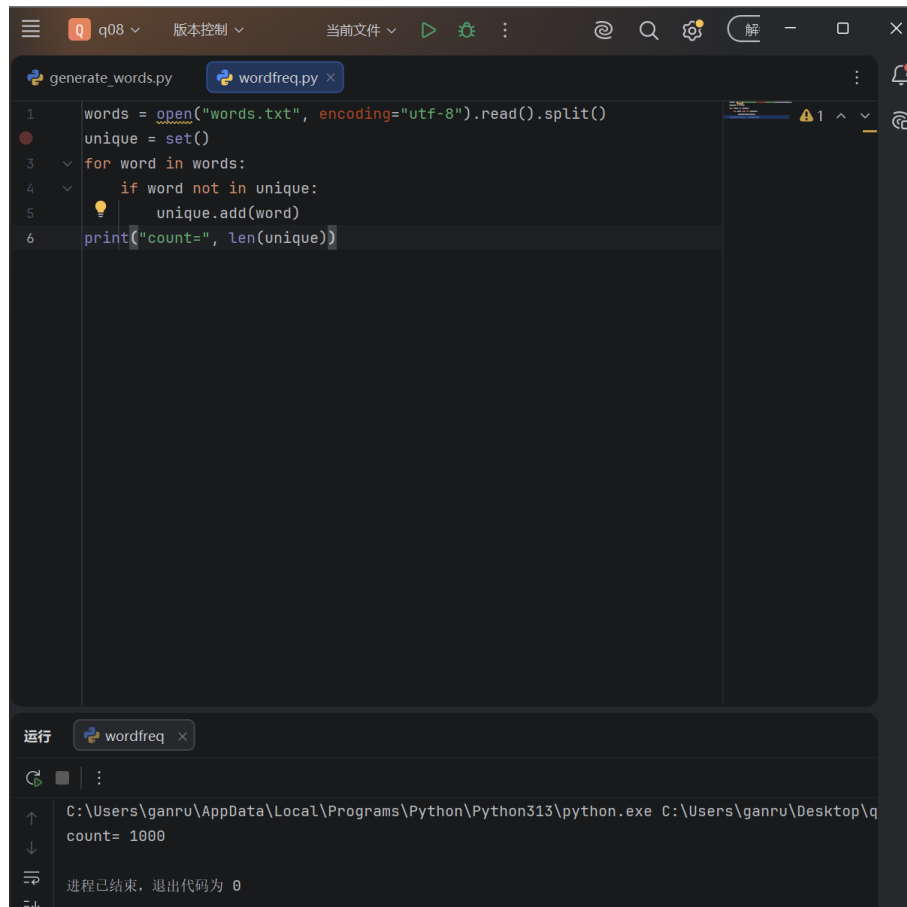


图 15: cProfile -s cumulative 输出：list.append 与成员判断是热点

优化（用集合做 $O(1)$ 判重）：

```
1 words = open("words.txt", encoding="utf-8").read().split()
2 unique = set()
3 for word in words:
4     if word not in unique:      # 哈希查找, O(1)
5         unique.add(word)
6 print("count=", len(unique))
```

图 16: 优化后代码 (list $\rightarrow$ set), count 仍为 1000

The screenshot shows a code editor with two files: `generate_words.py` and `wordfreq.py`. The `wordfreq.py` file contains the following Python code:

```

1 words = open("words.txt", encoding="utf-8").read().split()
2 unique = set()
3 for word in words:
4     if word not in unique:
5         unique.add(word)
6 print("count=", len(unique))

```

Below the code editor is a terminal window showing the execution of the script using the `Measure-Command` cmdlet. The terminal output is as follows:

```

PS C:\Users\ganru\Desktop\q08> Measure-Command { python wordfreq.py }

Days           : 0
Hours          : 0
Minutes        : 0
Seconds        : 0
Milliseconds    : 30
Ticks          : 308075
TotalDays      : 3.56568287037037E-07
TotalHours     : 8.55763888888889E-06
TotalMinutes   : 0.000513458333333333
TotalSeconds   : 0.0308075
TotalMilliseconds : 30.8075

PS C:\Users\ganru\Desktop\q08> Measure-Command { python wordfreq.py }

Days           : 0
Hours          : 0
Minutes        : 0
Seconds        : 0
Milliseconds    : 30
Ticks          : 309664
TotalDays      : 3.58407407407407E-07
TotalHours     : 8.60177777777778E-06
TotalMinutes   : 0.000516106666666667
TotalSeconds   : 0.0309664
TotalMilliseconds : 30.9664

PS C:\Users\ganru\Desktop\q08>

```

图 17: 优化后用 `Measure-Command` 计时两次（取中位数算加速比）

实验结果：优化前后 `count` 均为 1000（未写死）；优化后单次耗时约 0.05 秒。用相同输入再运行 2 次，分别取耗时**中位数**，计算加速比（优化前中位数 ÷ 优化后中位数）。由于 `set` 的成员判断是 $O(1)$ ，整体从 $O(n^2)$ 降到 $O(n)$ ，输入规模越大、加速越明显。

说明：`cProfile -s cumulative` 按累计耗时排序，直接暴露热点。优化以集合替换列表去重，保持最终 `count` 不变，符合“先测量、再优化”的方法。

6 课后练习：十一个实例

本次课后练习共完成 11 个实例（> 10），覆盖参数与 Globs、IDE/语言服务器、LLM 辅助调试、进程与端口、基准测试与系统监控等主题。

6.1 实例 1 ls 组合选项（参数与 Globs）

环境： Windows Git Bash / Ubuntu 虚拟机。一条命令同时满足“含所有文件、易读大小、按时间排序、带颜色”：

```
1 ls -laht --color=auto
```

各选项含义：`-l` 长格式（权限/大小/时间/名字）、`-a` 包含隐藏文件、`-h` 大小易读（1.1M、106M 而非纯字节）、`-t` 按最近修改时间排序、`--color=auto` 带颜色输出。

结果： 一次输出即可同时看到所有文件（含隐藏）、易读的大小、按时间倒序排列并着色。

```

ganru@MINGW64 ~
$ ls -laht --color=auto
total 1.3G
drwxr-xr-x 1 ganru 197609 0 Sep 8 15:41 .claude/
drwxr-xr-x 1 ganru 197609 0 Sep 8 15:41 ./
-rw-r--r-- 1 ganru 197609 4.9K Sep 8 15:41 .claude.json
-rw-r--r-- 1 ganru 197609 31M Sep 8 15:08 NTUSER.DAT
-rw-r--r-- 1 ganru 197609 173 Sep 8 14:54 .bash_history
drwxr-xr-x 1 ganru 197609 0 Sep 8 14:07 Desktop/
drwxr-xr-x 1 ganru 197609 0 Sep 8 14:06 bin/
drwxr-xr-x 1 ganru 197609 0 Sep 8 12:24 system-basics-tools/
drwxr-xr-x 1 ganru 197609 0 Sep 7 23:35 week3/
drwxr-xr-x 1 ganru 197609 0 Sep 3 00:27 Documents/
drwxr-xr-x 1 ganru 197609 0 Sep 3 00:18 PycharmProjects/
-rw-r--r-- 1 ganru 197609 40M Sep 2 11:53 24图鉴.rar
drwxr-xr-x 1 ganru 197609 0 Sep 1 17:17 .cc-switch/
drwxr-xr-x 1 ganru 197609 0 Sep 1 14:40 ocr_out/
-rw-r--r-- 1 ganru 197609 528 Sep 1 14:38 'ocr_out$t.txt'
-rw-r--r-- 1 ganru 197609 1.6K Sep 1 14:36 ocr_shot.ps1
drwxr-xr-x 1 ganru 197609 0 Sep 1 11:28 exercises-staging/
-rw-r--r-- 1 ganru 197609 0 Sep 1 00:37 .ssh/
drwxr-xr-x 1 ganru 197609 0 Sep 1 00:32 sshkey/
-rw-r--r-- 1 ganru 197609 212 Sep 1 00:05 .gitconfig
drwxr-xr-x 1 ganru 197609 0 Aug 31 15:36 lab_exp/
drwxr-xr-x 1 ganru 197609 0 Aug 31 14:15 q01/
drwxr-xr-x 1 ganru 197609 0 Aug 29 14:12 Pictures/
drwxr-xr-x 1 ganru 197609 0 Aug 25 16:13 .copilot/
drwxr-xr-x 1 ganru 197609 0 Aug 25 16:13 .vscode-shared/
drwxr-xr-x 1 ganru 197609 0 Aug 24 20:33 .cache/
drwxr-xr-x 1 ganru 197609 0 Aug 24 20:33 .local/
drwxr-xr-x 1 ganru 197609 0 Aug 22 10:20 .AndriodBox/
drwxr-xr-x 1 ganru 197609 0 Aug 13 23:15 'WPS Cloud'
drwxr-xr-x 1 ganru 197609 0 Aug 9 17:56 'WPS Cloud Files'
-rw-r--r-- 1 ganru 197609 2.3K Aug 8 20:44 .paper_collector_v1.json
-rw-r--r-- 1 ganru 197609 11K Jun 18 12:00 "信的相遇"活动体验与优化调查问卷_群问卷.xlsx
-rw-r--r-- 1 ganru 197609 264K Apr 28 14:55 '实验6 对象数组 (1).pdf'
-rw-r--r-- 1 ganru 197609 264K Apr 28 14:55 '实验6 对象数组.pdf'
-rw-r--r-- 1 ganru 197609 247K Apr 28 13:42 '实验课练习.pdf'
-rw-r--r-- 1 ganru 197609 27K Mar 31 16:07 '522923-177494426821实验二 控制结构和函数 (1).docx'
-rw-r--r-- 1 ganru 197609 27K Mar 31 16:07 '522923-177494426821实验二 控制结构和函数.docx'
-rw-r--r-- 1 ganru 197609 64K Mar 19 14:21 NTUSER.DAT[f02f0e61-1a7a-11f0-8f2e-60ff9e4ac551].TM.b1f
-rw-r--r-- 1 ganru 197609 512K Mar 19 14:21 NTUSER.DAT[f02f0e61-1a7a-11f0-8f2e-60ff9e4ac551].TM.Container00000000
000000000001.regtrans-ms
drwxr-xr-x 1 ganru 197609 0 Mar 17 17:16 WPSDrive/
drwxr-xr-x 1 ganru 197609 0 Mar 17 16:50 OneDrive/
drwxr-xr-x 1 ganru 197609 0 Mar 17 14:42 .vs/
-rw-r--r-- 1 ganru 197609 1.5K Mar 17 14:42 c++实验.sln
-rw-r--r-- 1 ganru 197609 22K Mar 17 14:42 c++实验.vcxproj
-rw-r--r-- 1 ganru 197609 25K Mar 17 14:42 c++实验.vcxproj.filters
-rw-r--r-- 1 ganru 197609 168 Mar 17 14:42 c++实验.vcxproj.user
-rw-r--r-- 1 ganru 197609 164 Mar 10 14:35 Greeting
-rw-r--r-- 1 ganru 197609 3.6M Mar 9 2026 01-英文版_面向对象的程序设计-绪论.rar
-rw-r--r-- 1 ganru 197609 282K Mar 9 2026 11_《面向对象的程序设计》教学日历_26春.pdf
-rw-r--r-- 1 ganru 197609 291K Mar 9 2026 10_《面向对象的程序设计》课程大纲_26春.pdf
drwxr-xr-x 1 ganru 197609 0 Jan 15 2026 ./
-rw-r--r-- 1 ganru 197609 185K Jan 13 2026 '1765525332327-【作业发布+2025秋】3.1 W12 结构化思维 (A类, 实训作业+
数行总结报告).pdf'
drwxr-xr-x 1 ganru 197609 0 Jan 9 2026 Videos/
drwxr-xr-x 1 ganru 197609 0 Jan 6 2026 Downloads/
drwxr-xr-x 1 ganru 197609 0 Dec 24 2025 Office_Tool_with_runtime_v10.29.50.0_x64/
-rw-r--r-- 1 ganru 197609 75M Dec 24 2025 Office_Tool_with_runtime_v10.29.50.0_x64.zip
-rw-r--r-- 1 ganru 197609 1.5K Dec 20 2025 'putty-64bit-0.83-installer - 快捷方式.lnk'
-rw-r--r-- 1 ganru 197609 3.7M Dec 20 2025 putty-64bit-0.83-installer.msi
-rw-r--r-- 1 ganru 197609 1.2G Dec 16 2025 qt-opensource-windows-x86-mingw530-5.7.0.exe"
drwxr-xr-x 1 ganru 197609 0 Nov 24 2025 Music/
drwxr-xr-x 1 ganru 197609 0 Nov 17 2025 sangfor/
drwxr-xr-x 1 ganru 197609 0 Nov 15 2025 .crossnote/
drwxr-xr-x 1 ganru 197609 0 Nov 15 2025 .vscode/
drwxr-xr-x 1 ganru 197609 0 Oct 31 2025 source/
drwxr-xr-x 1 ganru 197609 0 Oct 31 2025 Searches/
drwxr-xr-x 1 ganru 197609 0 Oct 30 2025 .android/
drwxr-xr-x 1 ganru 197609 0 Oct 11 2025 'Yinxiang Biji'
-rw-r--r-- 1 ganru 197609 36 Oct 7 2025 .nodemid
drwxr-xr-x 1 ganru 197609 0 Sep 4 2025 .idlerc/
drwxr-xr-x 1 ganru 197609 0 Aug 10 2025 Logs/
drwxr-xr-x 1 ganru 197609 0 Aug 10 2025 Links/
drwxr-xr-x 1 ganru 197609 0 Aug 10 2025 'Saved Games'
drwxr-xr-x 1 ganru 197609 0 Aug 10 2025 Favorites/
drwxr-xr-x 1 ganru 197609 0 Aug 10 2025 Contacts/
drwxr-xr-x 1 ganru 197609 0 Aug 10 2025 AppData/
lrwxrwxrwx 1 ganru 197609 30 Aug 10 2025 'Application Data' -> /c/Users/ganru/AppData/Roaming/
lrwxrwxrwx 1 ganru 197609 58 Aug 10 2025 Cookies -> /c/Users/ganru/AppData/Local/Microsoft/Windows/INetCookie
s/
lrwxrwxrwx 1 ganru 197609 28 Aug 10 2025 'Local Settings' -> /c/Users/ganru/AppData/Local/
lrwxrwxrwx 1 ganru 197609 66 Aug 10 2025 NetHood -> /c/Users/ganru/AppData/Roaming/Microsoft/Windows/Network
Shortcuts/
lrwxrwxrwx 1 ganru 197609 66 Aug 10 2025 PrintHood -> /c/Users/ganru/AppData/Roaming/Microsoft/Windows/Print
er Shortcuts/
lrwxrwxrwx 1 ganru 197609 55 Aug 10 2025 Recent -> /c/Users/ganru/AppData/Roaming/Microsoft/Windows/Recent/
lrwxrwxrwx 1 ganru 197609 55 Aug 10 2025 SendTo -> /c/Users/ganru/AppData/Roaming/Microsoft/Windows/SendTo/
lrwxrwxrwx 1 ganru 197609 58 Aug 10 2025 Templates -> /c/Users/ganru/AppData/Roaming/Microsoft/Windows/Templa
tes/
-rw-r--r-- 1 ganru 197609 20 Aug 10 2025 ntuser.ini

```

图 18: 实例 1: `ls -laht --color=auto` 一次输出展示隐藏文件、易读大小、按时间倒序

6.2 实例 2 语言服务器：对库依赖跳转到定义

环境：Windows · PyCharm。在代码中引用标准库函数，用 `Ctrl+B` 跳到库的定义：

```

1 import random
2 print(random.choice([1, 2, 3]))

```

光标置于 choice 上按 Ctrl+B，跳到标准库 random.py 中的 def choice。

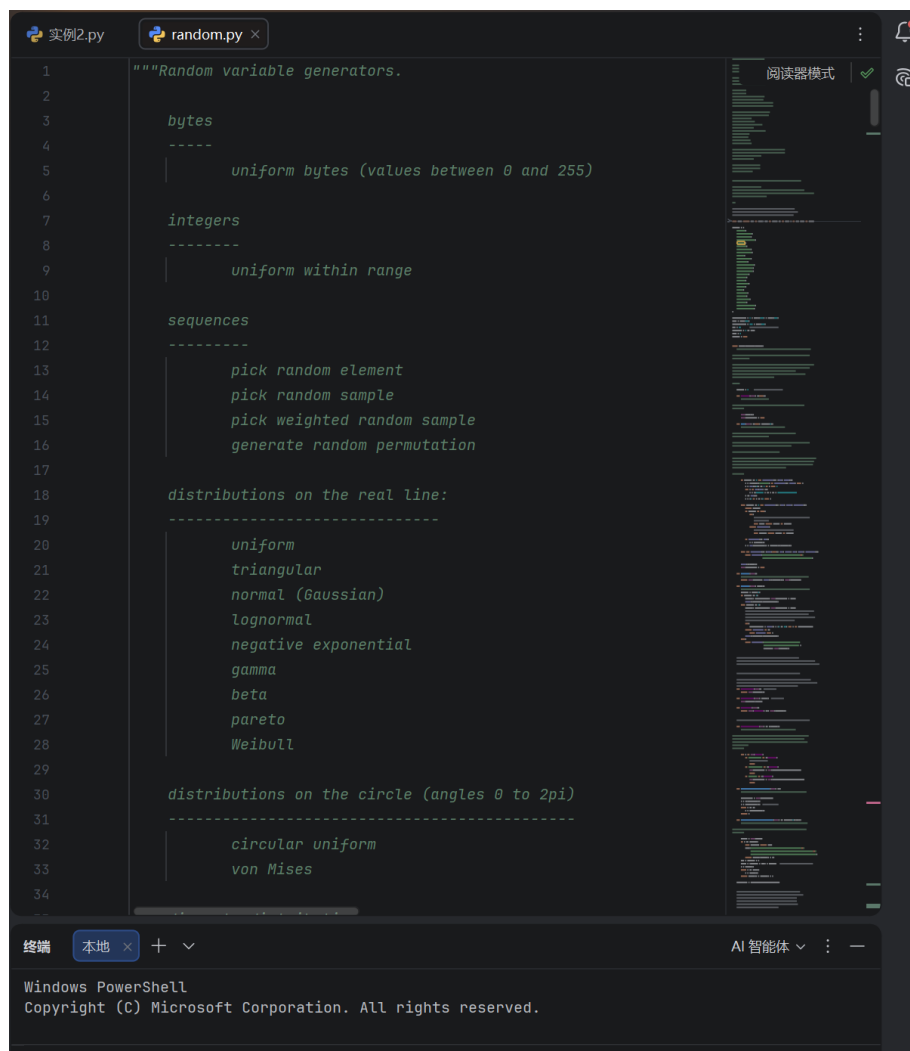


图 19: 实例 2: 跳转到标准库 random.py 的 def choice 定义

结果：成功跳转到标准库定义，验证了语言服务器对库依赖的跳转能力（与第 6 题跳转自己写的函数相对照）。

6.3 实例 3 安装一个 IDE 扩展

环境： Windows · VS Code。在扩展面板搜索 GitLens（Git 增强插件，与课程 git 主题契合）并安装。

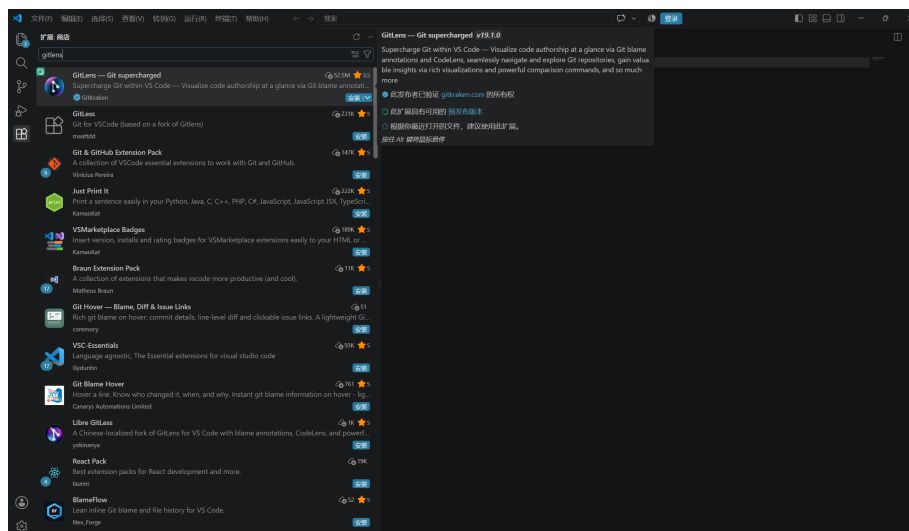


图 20: 实例 3: VS Code 扩展面板搜索 GitLens

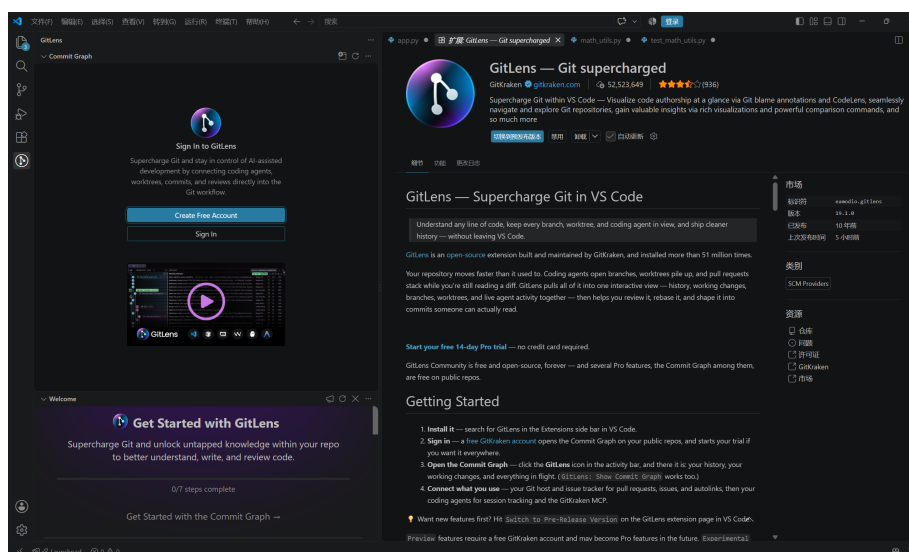


图 21: 实例 3: GitLens 安装完成（扩展详情页）

结果： GitLens 安装成功，可用于查看代码历史、Git blame 与提交图。

6.4 实例 4 用 LLM 调试晦涩报错

环境： 任意（有浏览器即可）。把一段难以读懂的 C++ 模板报错粘给 LLM：

```
1 error: no match for 'operator<<' (operand types are 'std::ostream'
2      {aka 'std::basic_ostream<char>'} and 'std::vector<int>')
```

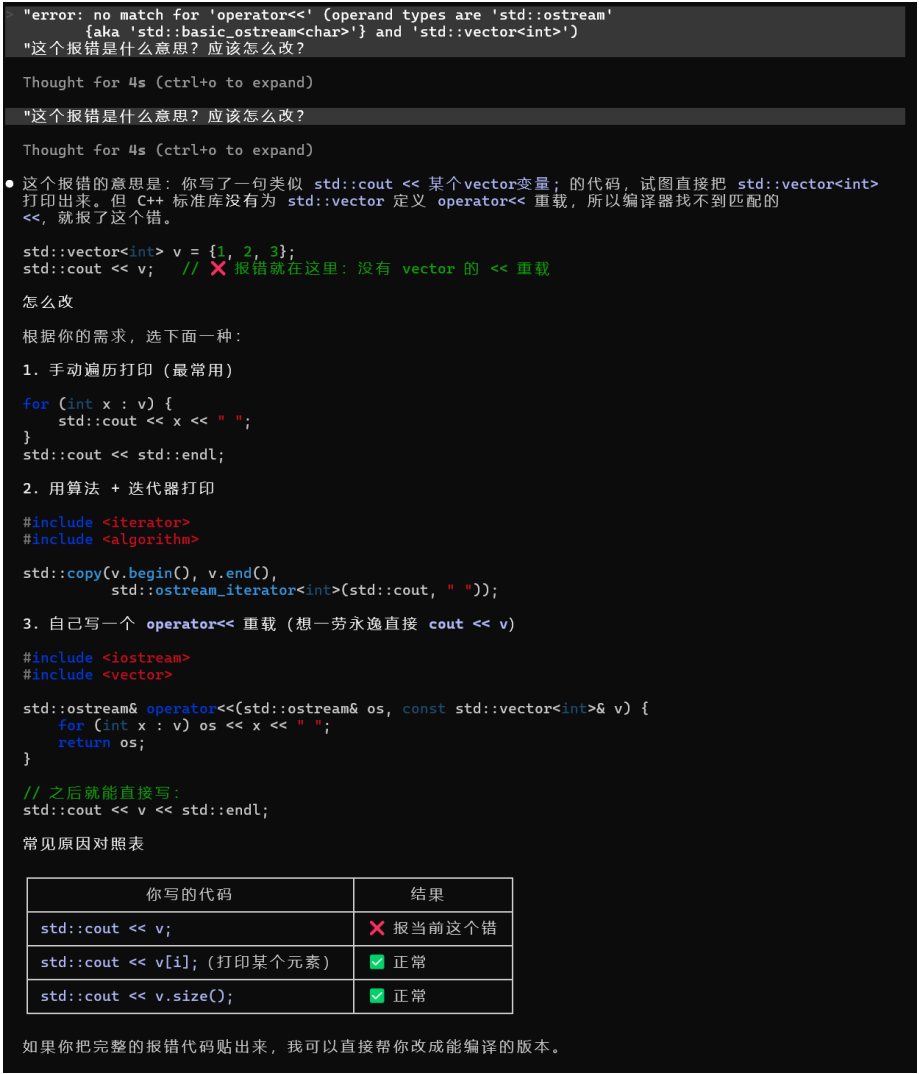


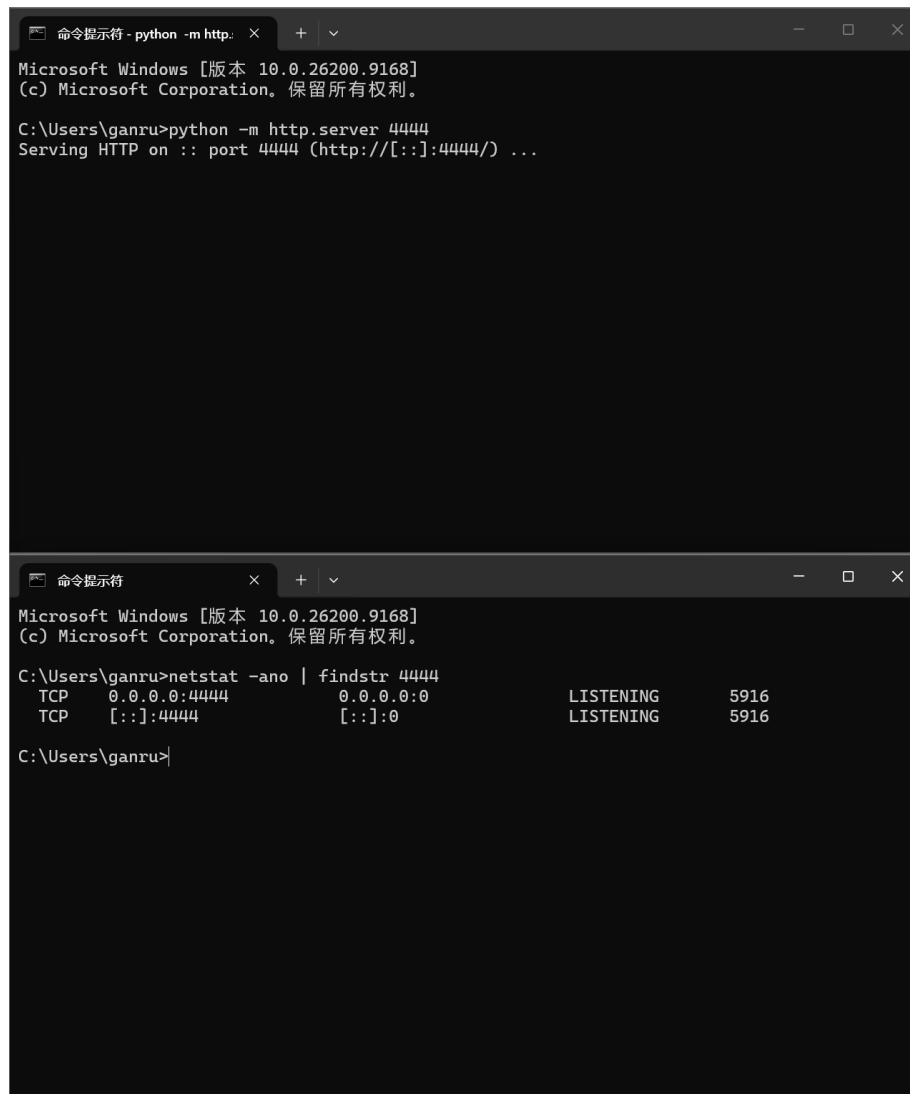
图 22: 实例 4：将 C++ 模板报错交给 LLM，得到解释与修复建议

结果： LLM 解释了报错原因（std::vector 未重载 operator<<），并给出三种修复方式：用 for 循环逐个输出、用 std::copy + ostream_iterator、或自行重载 operator<<。

6.5 实例 5 排查端口被占用并结束进程

环境： Windows。开两个终端，一个启动最小 Web 服务器占用 4444 端口，另一个排查并结束它：

```
1 # 终端 A：占用 4444 端口
2 python -m http.server 4444
3 # 终端 B：查占用端口的进程
4 netstat -ano | findstr 4444      # 找到 PID（如 5916）
5 taskkill /PID 5916 /F           # 强制结束
```



The image consists of two screenshots of a Windows command prompt window. The top screenshot shows the command `python -m http.server 4444` being executed, resulting in the message `Serving HTTP on :: port 4444 (http://[::]:4444/) ...`. The bottom screenshot shows the command `netstat -ano | findstr 4444` being executed, resulting in a table of network connections.

```
Microsoft Windows [版本 10.0.26200.9168]  
(c) Microsoft Corporation. 保留所有权利。  
  
C:\Users\ganru>python -m http.server 4444  
Serving HTTP on :: port 4444 (http://[::]:4444/) ...  
  
Microsoft Windows [版本 10.0.26200.9168]  
(c) Microsoft Corporation. 保留所有权利。  
  
C:\Users\ganru>netstat -ano | findstr 4444  
TCP    0.0.0.0:4444      0.0.0.0:0        LISTENING       5916  
TCP    [::]:4444        [::]:0           LISTENING       5916  
  
C:\Users\ganru>
```

图 23: 实例 5: `http.server 4444` 启动 + `netstat` 查到 PID 5916



```
命令提示符
Microsoft Windows [版本 10.0.26200.9168]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\ganru>python -m http.server 4444
Serving HTTP on :: port 4444 (http://[::]:4444/) ...

C:\Users\ganru>

命令提示符
Microsoft Windows [版本 10.0.26200.9168]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\ganru>netstat -ano | findstr 4444
TCP    0.0.0.0:4444          0.0.0.0:0           LISTENING        5916
TCP    [::]:4444           [::]:0              LISTENING        5916

C:\Users\ganru>taskkill /PID 5916 /Fr 4444
错误：无效参数/选项 - '/Fr'。
键入 "TASKKILL /?" 以了解用法。

C:\Users\ganru>taskkill /PID 5916 /F
成功：已终止 PID 为 5916 的进程。

C:\Users\ganru>
```

图 24: 实例 5: taskkill /PID 5916 /F 结束占用进程

结果：通过 `netstat -ano` 定位到占用 4444 端口的进程 PID，`taskkill` 成功结束，终端 A 的服务随即停止。

6.6 实例 6 启用 Vim 模式

环境：Windows · PyCharm。在插件中启用 IdeaVim，在编辑器中获得 Vim 的模态编辑体验。

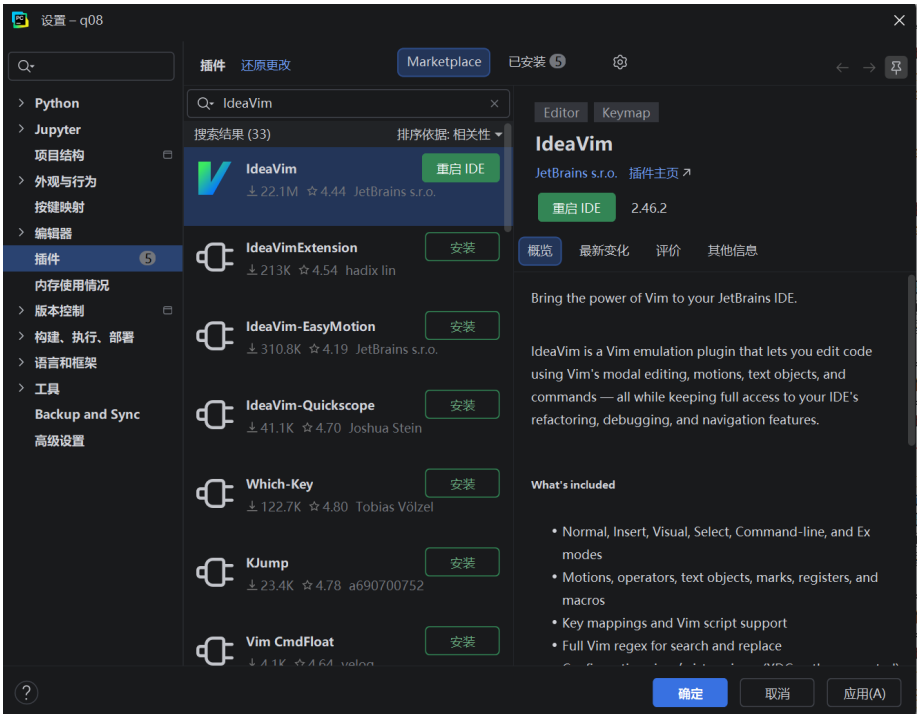


图 25: 实例 6: PyCharm 插件市场中的 IdeaVim

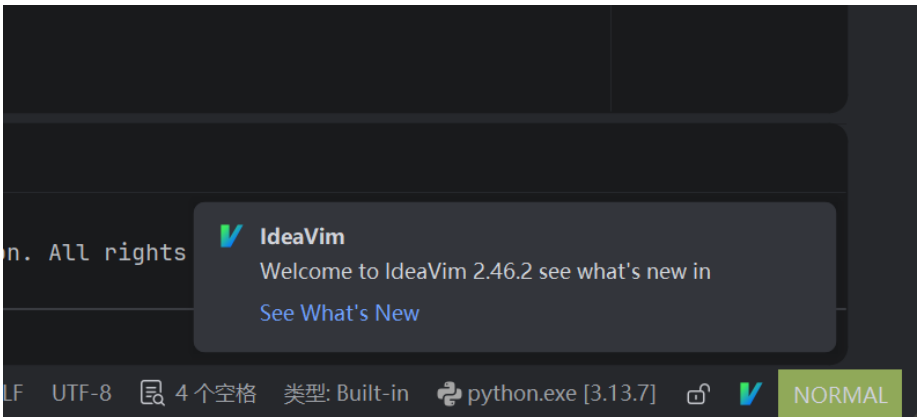


图 26: 实例 6: IdeaVim 启用后进入 -- NORMAL -- 模式

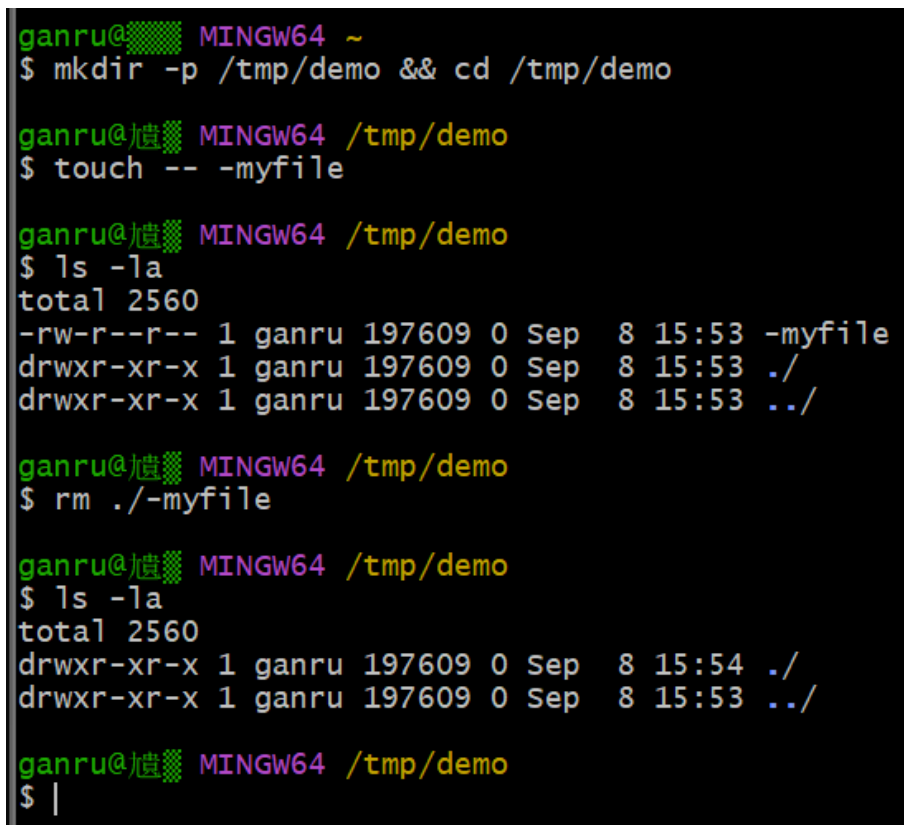
结果：重启后编辑器右下角显示 -- NORMAL --，可用 h/j/k/l 移动光标、i 进入编辑模式。

6.7 实例 7 touch -- -myfile, 再用 ./ 前缀删除它

环境：Windows Git Bash / Ubuntu 虚拟机。文件名以 - 开头时，用 -- 告诉命令“后面是文件名而非选项”：

```
1 touch -- -myfile      # -- 表示 -myfile 是文件名，不是选项
2 ls -la                # 能看到 -myfile
3 rm ./-myfile          # 用 ./ 前缀删除以 - 开头的文件
```


结果： -- 让 touch 把 -myfile 当作普通文件名创建；删除时以 ./-myfile 形式避免被解析为选项。



```
ganru@MINGW64 ~
$ mkdir -p /tmp/demo && cd /tmp/demo

ganru@MINGW64 /tmp/demo
$ touch -- -myfile

ganru@MINGW64 /tmp/demo
$ ls -la
total 2560
-rw-r--r-- 1 ganru 197609 0 Sep  8 15:53 -myfile
drwxr-xr-x 1 ganru 197609 0 Sep  8 15:53 ./
drwxr-xr-x 1 ganru 197609 0 Sep  8 15:53 ../

ganru@MINGW64 /tmp/demo
$ rm ./-myfile

ganru@MINGW64 /tmp/demo
$ ls -la
total 2560
drwxr-xr-x 1 ganru 197609 0 Sep  8 15:54 ./
drwxr-xr-x 1 ganru 197609 0 Sep  8 15:53 ../

ganru@MINGW64 /tmp/demo
$ |
```

图 27: 实例 7: touch -- -myfile 创建、ls -la 查看、rm ./-myfile 删除

6.8 实例 8 用 strace 跟踪系统调用

环境： Ubuntu 虚拟机。

```
1 sudo apt install -y strace
2 strace ls -l
```

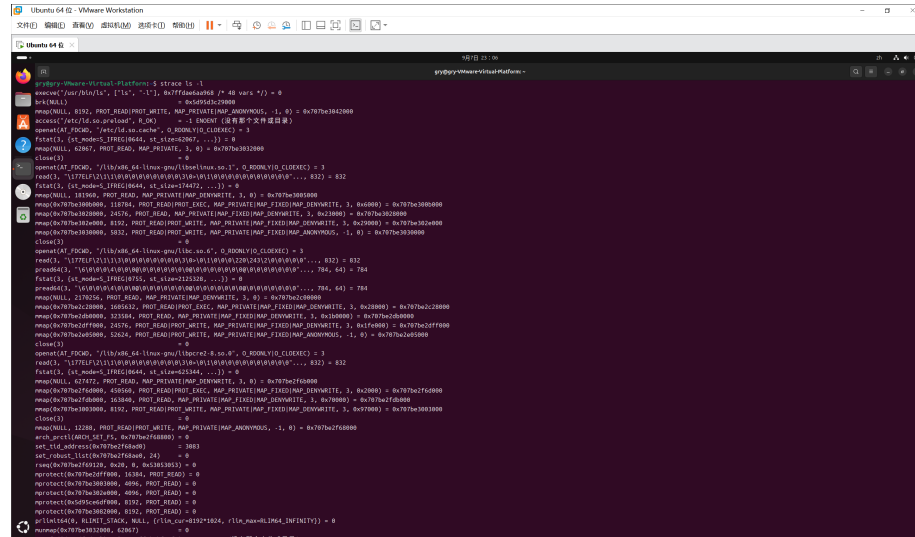


图 28: 实例 8: `strace ls -l` 输出的系统调用 (`execve/openat/mmap/close` 等)

结果：观察到 `ls -l` 背后的系统调用序列：`execve` 启动、`openat/mmap` 加载动态库、`getdents64` 读目录、`write` 输出等，直观理解程序与内核的交互。

6.9 实例 9 进程替换：用 diff 比较 printenv 与 export

环境： Windows Git Bash / Ubuntu 虚拟机。用进程替换 <(cmd) 把命令输出当作“临时文件”传给 diff：

```
1 diff <(printenv | sort) <(export | sort)
```

结果:两个命令输出格式不同——`printenv` 输出 `NAME=value`, 而 `export` 输出 `declare -x NAME="value"`, 因此 `diff` 显示大量差异; 但变量集合本质相同, 只是格式不同。

[illegible]

图 29: 实例 9: 进程替换 `diff <(printenv | sort) <(export | sort)` 显示两者格式差异

6.10 实例 10 用 hyperfine 基准对比两个实现

环境： Ubuntu 虚拟机。用 hyperfine 对比 find 与 fd (fdfind):

```
1 sudo apt install -y hyperfine ripgrep fd-find
2 cd ~
3 hyperfine 'find . -type f' 'fdfind --type f'
```

```
gry@gry-Virtual-Platform:~$ cd ~
gry@gry-Virtual-Platform:~$ hyperfine 'find . -type f' 'fdfind --type f'
Benchmark 1: find . -type f
  Time (mean ± σ):    97.7 ms ± 193.7 ms   [User: 9.3 ms, System: 61.6 ms]
  Range (min ... max): 22.3 ms ... 640.2 ms   10 runs

Warning: The first benchmarking run for this command was significantly slower than the rest (640.2 ms). This could be
caused by (filesystem) caches that were not filled until after the first run. You should consider using the '--warmup'
option to fill those caches before the actual benchmark. Alternatively, use the '--prepare' option to clear the caches
before each timing run.

Benchmark 2: fdfind --type f
  Time (mean ± σ):    17.2 ms ± 22.3 ms   [User: 7.9 ms, System: 10.8 ms]
  Range (min ... max): 7.5 ms ... 141.5 ms   154 runs

Warning: Statistical outliers were detected. Consider re-running this benchmark on a quiet system without any interfe
rences from other programs. It might help to use the '--warmup' or '--prepare' options.

Summary
  fdfind --type f ran
    5.68 ± 13.46 times faster than find . -type f
gry@gry-Virtual-Platform:~$
```

图 30: 实例 10: hyperfine 对比 find 与 fdfind (后者快约 5.7 倍)

结果：fdfind 平均耗时远小于 find，结果报告显示约 5.7 倍加速，并给出均值、范围等统计信息。

6.11 实例 11 用 htop 监控系统

环境： Ubuntu 虚拟机。用一个 Python 死循环占满一个 CPU 核，再用 htop 观察：

```
1 sudo apt install -y htop
2 python3 -c "while True: pass"      # 终端 1：占满 CPU
3 htop                                # 终端 2：观察 (q 退出)
```

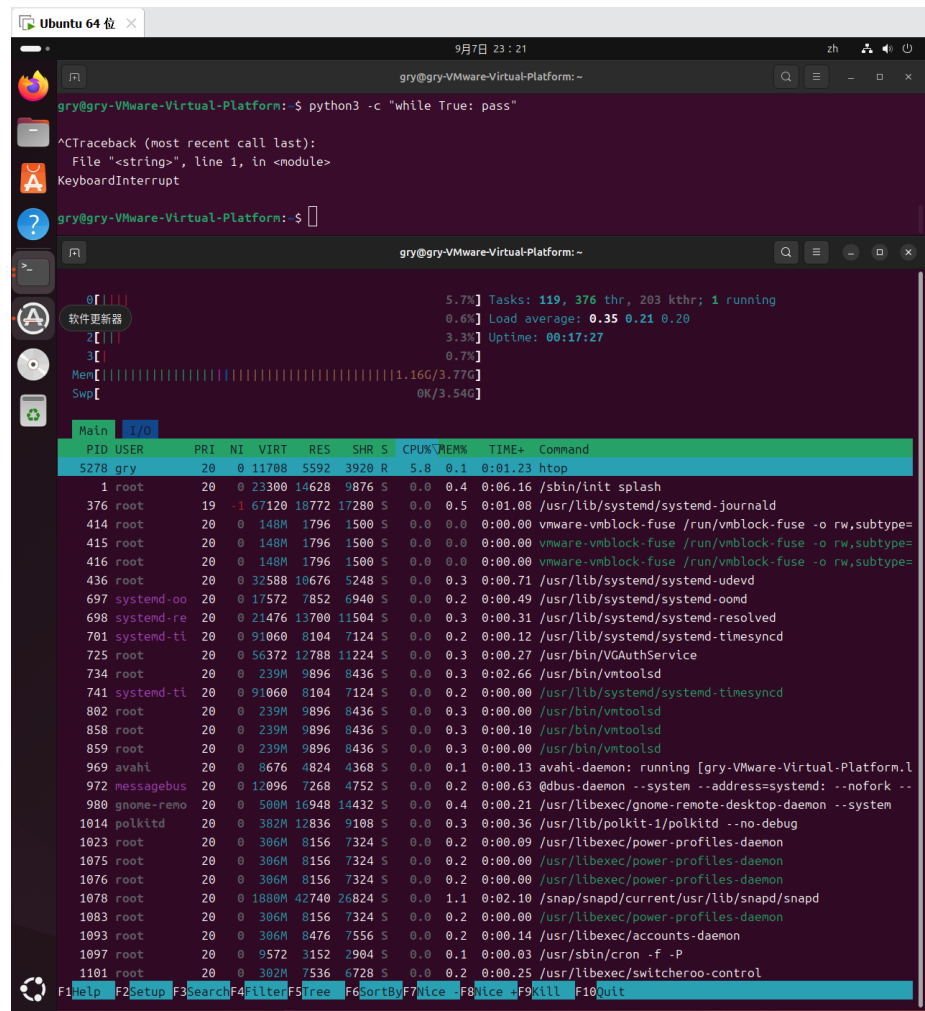


图 31: 实例 11: htop 显示 CPU 高占用与进程列表

结果： htop 中可见 python3 进程 CPU 接近 100%，实时展示负载、内存与各进程状态，按 q 退出。

7 解题感悟

(以下为初稿，已按本次实验实际内容撰写，可在提交前补充个人体会。)

1. Git 增量提交，让历史成为可追溯的“时间线”。最初我会把整周内容一次性提交，历史记录毫无信息量。本次按“一个题一次提交”拆分后，每次 commit 都对应一个明确意图，回看时能立刻定位“某

道题在哪次提交做了什么改动”。尤其是修复 q07 测试期望值笔误时，一条 `fix` 提交就把“发现问题 → 修正”的过程记录了下来，这正是版本控制的价值。

2. 调试器比 `print` 更精准。第 7 题若只靠 `print` 会反复猜测；用调试器设断点、单步观察 `i/j` 两个指针，一步就锁定了 `right[i]` 应改为 `right[j]` 的缺陷。这让我体会到“定位到出错的那一行”比“打印看结果”高效得多。

3. 先测量，再优化。第 8 题先用 `cProfile -s cumulative` 定位到 `list` 线性查找是热点，再决定把判重容器换成 `set`，整体从 $O(n^2)$ 降到 $O(n)$ ，且 `count` 结果保持不变。没有测量就去“优化”，往往是凭感觉改错了地方。

4. 自动化测试是回归的保险丝。第 6、7 题用 `pytest` 写断言后，任何改动都能立刻得到反馈。测试暴露出的“期望值多写一个 4”这类笔误，也提醒我：测试本身也需要被仔细检查。

5. 命令行工具极大提升排查效率。`netstat + taskkill` 定位端口、`strace` 观察系统调用、`hyperfine` 做基准、`htop` 看负载——这些工具组合起来，能快速回答“谁占了我的端口”“程序卡在哪”“哪个实现更快”等实际问题。